hashlock.

Security Audit

NOTE Protocol (DeFi)

November 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

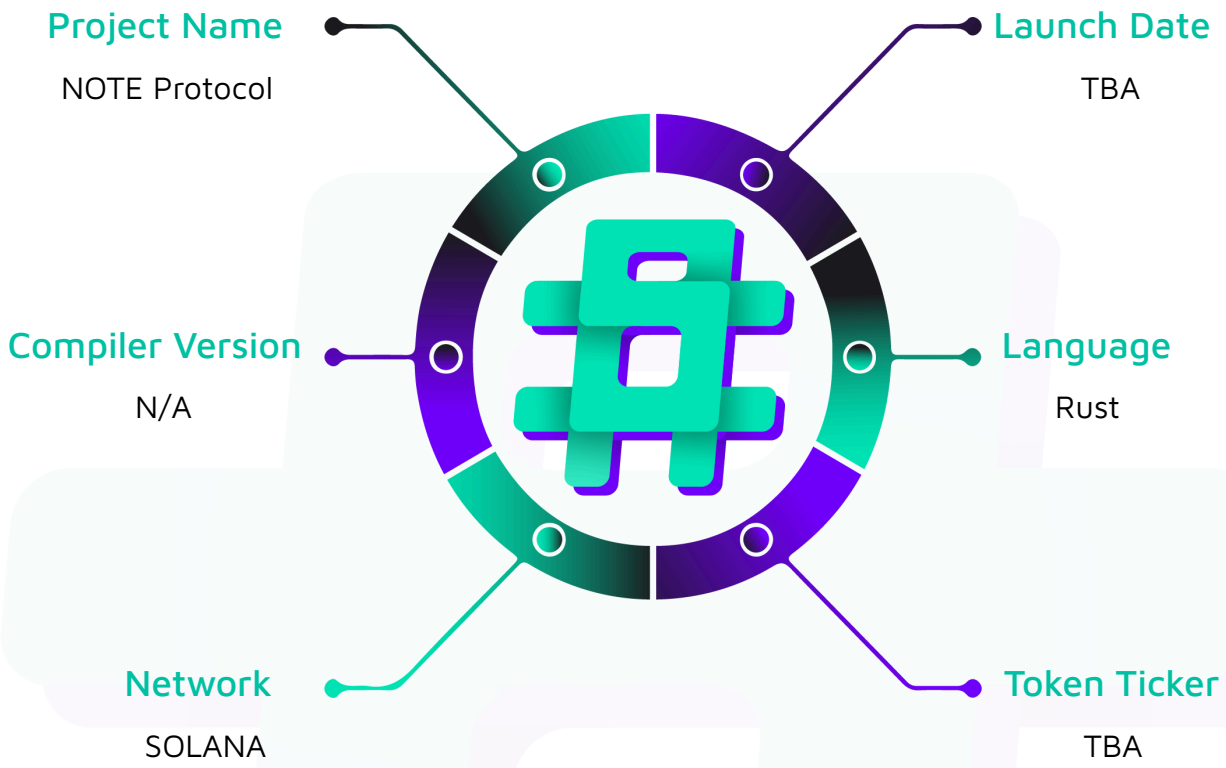
The NOTE Protocol team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

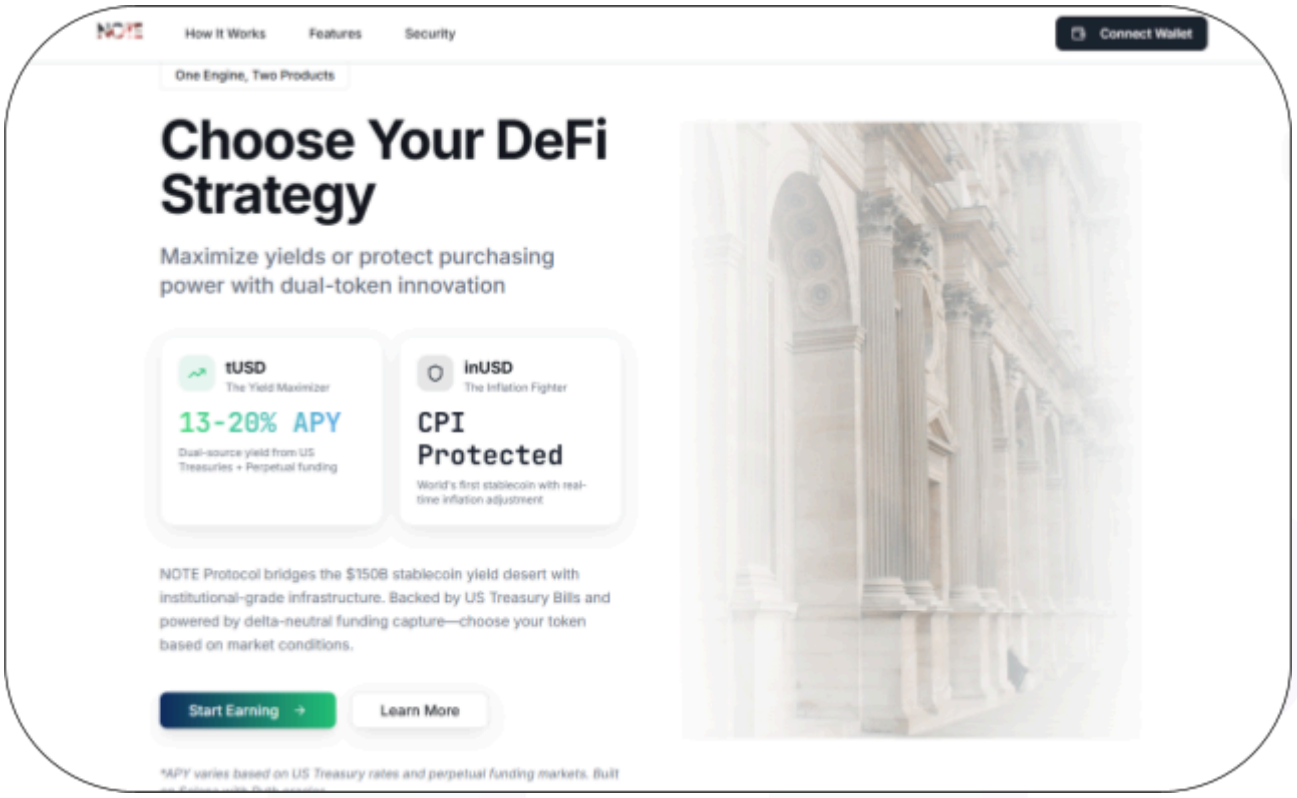
The NOTE Protocol is a DeFi platform built on the Solana blockchain, designed to create a secure and automated ecosystem for managing and generating yield from digital assets, with a focus on stablecoins such as USDC, USDT, and USDY. The associated website and infrastructure are being developed to provide a transparent and auditable interface, featuring full integrations with protocols like Drift, Jupiter, and oracle systems such as Pyth. The detailed documentation and audit workflow indicate that the project is in its final testing phase before the mainnet launch.

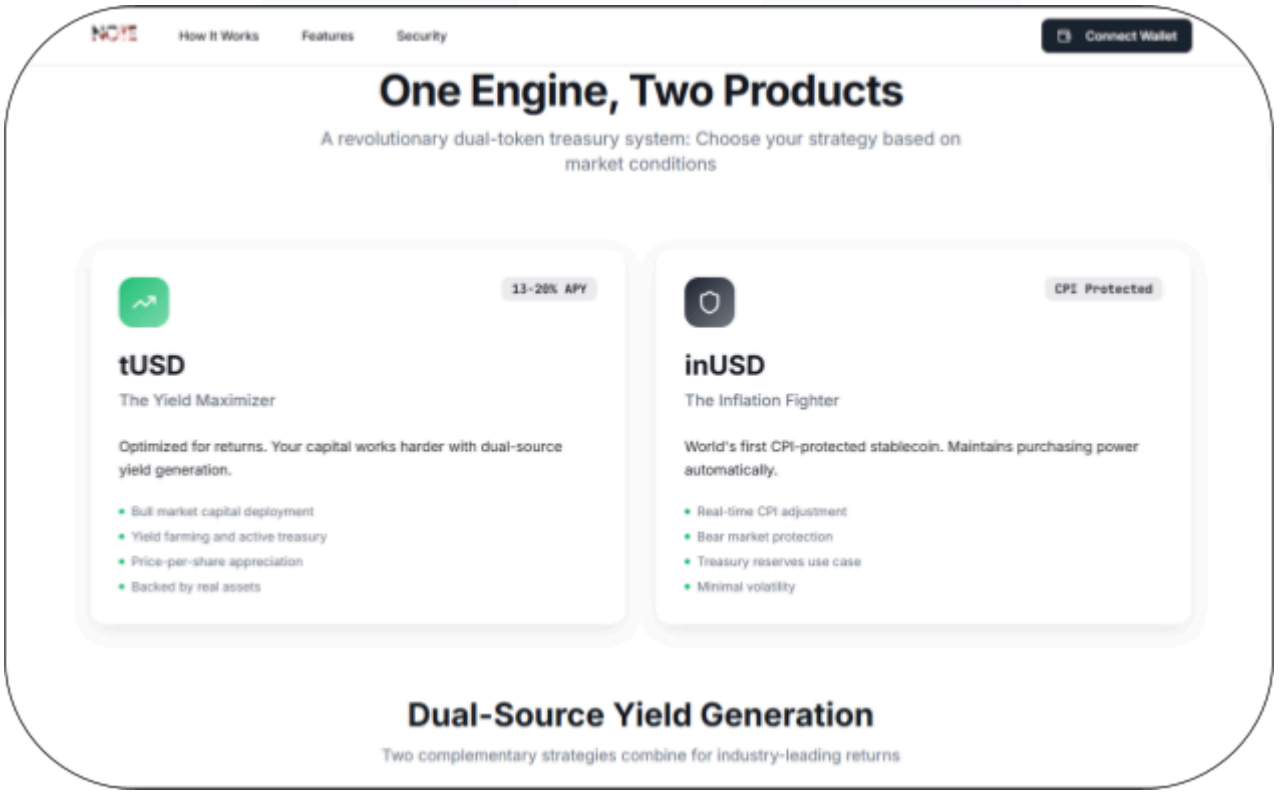
Project Name: NOTE Protocol
Project Type: Defi
Compiler Version: ^0.8.42
Website: www.NOTE Protocol.io
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the Rust code within the NOTE Protocol project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**NOTE Protocol Protocol Smart Contracts**
Platform	**Solana / Rust**
Audit Date	**November, 2025**
GitHub repo	https://github.com/Gorham-Research/NOTE-Complete
Contracts in scope	programs/hedge_manager programs/treasury_vault
Audited GitHub Commit Hash	94fa3ba6172ff7ea4abd7394b3625bbc8ee9e4a3
Fix Review GitHub Commit Hash	b82c0244b5ffa39adc1c2ce841207b380e8b1b3b

Note: *Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.*

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

14 High severity vulnerabilities

3 Medium severity vulnerabilities

2 Low severity vulnerabilities

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
programs/hedge_manager - Initialize Hedge state. - Manages perpetual futures positions. - Harvest fundings. - Close positions. - Set paused state.	**Contract achieves this functionality.**
programs/treasury_vault - Initializes the vault. - Allows users to deposit funds. - Withdraw funds and process withdrawals. - Rebalances portfolio. - Increase or decrease hedge positions. - Update protocol configurations. - Emergency shutdown mechanisms. - Integrate with third-party protocols, such as Jupiter, Ondo Finance, and Drift Protocol. - Query functionalities for the protocol.	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the NOTE Protocol project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the NOTE Protocol project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] programs/hedge_manager/src/lib.rs#InitializeHedge - Non-PDA HedgeState allows unauthorized state creation

Description

The `HedgeState` account is initialized as a regular account rather than a Program Derived Address (PDA). The `InitializeHedge` context uses `#[account(init, payer = authority, space = ...)]` without `seeds` and `bump` constraints, meaning anyone can call `initialize_hedge_state` with any keypair to create arbitrary `HedgeState` accounts.

This completely bypasses program integrity as there is no deterministic derivation linking the hedge state to the vault.

Impact

Critical integrity failure. For example, attackers can create fake `InitializeHedge` accounts that include arbitrary values, reference unauthorized hedge states in instructions, and eventually bypass the intended program logic by using malicious state accounts.

Recommendation

Convert `HedgeState` to a PDA instead of a regular account.

```
#[account(

    init,

    payer = authority,

    space = 8 + std::mem::size_of::<HedgeState>() + 128,

    seeds = [b"hedge_state"],

    bump
```

```
)]

pub hedge_state: Account<'info, HedgeState>,
```

Status

Resolved

[H-02] programs/hedge_manager/src/lib.rs - Missing signer validation allows unauthorized operations

Description

Multiple critical instruction contexts lack signer requirements, allowing anyone to execute privileged operations without authorization. The affected contexts are:

- `UpdatePosition` - used by `emergency_close_position` and `set_paused`.
- `DriftUpdatePosition` - used by `increase_position` and `decrease_position`.
- `HarvestFunding` - not used currently.
- `DriftHarvestFunding` - used by `harvest_funding`.

None of these contexts includes an `authority` or `vault_authority` signer validation. Anyone can call these functions to manipulate hedge positions, pause the system, or harvest funds.

Impact

- Unauthorized users can arbitrarily increase or decrease positions.
- Attackers can pause the hedge manager to cause a denial-of-service attack.
- Anyone can trigger emergency position closures.
- Unrestricted harvesting of funding payments.
- Total compromise of the hedging strategy.

Recommendation

Implement authority validations for all the contexts mentioned above, and add the corresponding validation in the instruction handlers.

Status

Resolved

[H-03] programs/hedge_manager/src/lib.rs - Hardcoded Oracle prices and TODOs result in unimplemented critical functionalities

Description

Multiple critical functions contain placeholder implementations and TODO comments indicating unfinished integration with Drift Protocol. For example, the `increase_position`, `decrease_position`, and `harvest_funding` functions contains the following comments:

- `// TODO: Implement actual Drift CPI call with oracle-validated price`
- `// TODO: Implement actual Drift CPI call with oracle validation`
- `// TODO: Implement actual Drift CPI call`

Additionally, the program uses hardcoded oracle prices and simulates operations that should execute real on-chain transactions. Specifically, `increase_position` and `decrease_position` both use hardcoded oracle prices that always validate successfully, completely bypassing real price feeds.

```
let oracle_price: i64 = 100_000_000;

let oracle_validation = OracleValidationResult {

    price: oracle_price,

    confidence: 95,

    is_valid: true,

};
```

Impact

- Positions are only tracked locally, but never opened or closed under Drift protocol.
- Oracle price manipulations are undetectable.
- Funding harvests don't settle real PnL from Drift positions.
- All positions and funding accounting exist only in the program state, with no real backing.

Recommendation

1. Implement real Oracle integration using Drift's Oracle accounts.
2. Implement all Drift CPI calls for position management.
3. Implement real PnL settlement in `harvest_funding`.
4. Remove all placeholders and simulations before mainnet deployment.

Status

Resolved

[H-04] programs/hedge_manager/src/lib.rs - Incorrect vault signer implementation using mutable SystemAccount

Description

The program uses an incorrect pattern for vault signing authority by including a separate `vault_signer` account that is defined as a mutable `SystemAccount`:

```
#[account(
    mut,
    seeds = [b"vault_signer", vault.key().as_ref()],
    bump
)]
pub vault_signer: SystemAccount<'info>,
```

This implementation is incorrect because it creates an unnecessary separate signer account. If the vault itself needs to sign operations, it should be implemented as a PDA directly rather than requiring a separate derived signer.

Additionally, the actual vault is defined as an `AccountInfo` without PDA constraints, meaning users can initialize it more than once. This allows attackers to create arbitrary "vault signers" for fake vaults, as mentioned in [H-01].

Impact

This causes confusion between the signing authority and account ownership. As shown in H-01, the entire signing scheme is compromised.

Recommendation

Remove the separate `vault_signer` account and implement the vault itself as a PDA with proper signing authority:

```
#[account(
    seeds = [b"vault"],
    bump = vault.bump
```

```
)]

pub vault: Account<'info, VaultState>,
```

When performing CPIs that require vault authority, use the vault's PDA seeds directly:

```
let vault_seeds = &[

    b"vault",

    &[vault.bump]

];

let signer_seeds = &[&vault_seeds[..]];



// Use in CPI

ctx.accounts.into_drift_context().with_signer(signer_seeds)
```

Status

Resolved

[H-05] programs/treasury_vault/src/lib.rs#withdraw_to_usdy, withdraw_to_fobxx - Passing fake token mint allows complete protocol drainage

Description

The withdrawal functions allow users to pass any token mint as `tusd_mint` without validation against the vault's stored mint address. This allows an attacker to:

1. Create their own worthless token mint
2. Mint millions of fake tokens for themselves
3. Call withdraw functions passing their fake mint as `tusd_mint`
4. Burn worthless tokens while receiving real USDC/USDY/FOBXX

Impact

Complete drainage of the vault's collateral. Attackers can steal all USDC, USDY, and FOBXX tokens from the vault by burning worthless tokens.

Recommendation

Implement validation before burn operations:

```
require!(ctx.accounts.tusd_mint.key() == v.tusd_mint, ErrorCode::Unauthorized);
```

Status

Resolved

[H-06] programs/treasury_vault/src/lib.rs#initialize_vault - Missing fee custody initialization causes denial of service

Description

The `fee_custody` field is required in multiple contexts, but is never initialized in `initialize_vault`:

```
// In initialize_vault - fee_custody is NOT set

v.collateral_mint = ctx.accounts.collateral_mint.key();

v.collateral_custody = ctx.accounts.collateral_custody.key();
```

This means functions that validate `fee_custody` to be the expected address will always fail:

```
// deposit, deposit_tusd, deposit_inusd

require!(v.fee_custody == ctx.accounts.fee_custody.key(), ErrorCode::Unauthorized);
```

Additionally, the fees are not minted to the `v.fee_custody` in the `deposit`, `deposit_tusd`, and `deposit_inusd` functions, despite the validation.

Impact

All deposit operations requiring `fee_custody` validation will fail.

Recommendation

1. Initialize `fee_custody` in `initialize_vault`.
2. Mint the fees to the `v.fee_custody` address in the `deposit`, `deposit_tusd`, and `deposit_inusd` functions.

Status

Resolved

[H-07] programs/treasury_vault/src/lib.rs#deposit_usdy, deposit_fobxx - Missing fee custody validation causes loss of protocol fees

Description

The `deposit_usdy` and `deposit_fobxx` functions mint the fee to the fee custody account, as determined by `fee_amount`.

The issue is that the functions do not validate that the user-supplied account (`ctx.accounts.fee_custody`) equals the whitelisted fee custody address stored in the vault (`v.fee_custody`).

Impact

The protocol will lose fees if the user supplies `ctx.accounts.fee_custody` as their own address.

Recommendation

Implement the following validation:

```
require!(v.fee_custody == ctx.accounts.fee_custody.key(), ErrorCode::Unauthorized);
```

Status

Resolved

[H-08] programs/treasury_vault/src/lib.rs#is_whitelisted_collateral - Devnet mint in production whitelist

Description

The `is_whitelisted_collateral` function includes a devnet USDC mint in the production whitelist without any environment check:

```
pub fn is_whitelisted_collateral(mint: &Pubkey) -> bool {
    // Mainnet stablecoins
    *mint == USDC_MAINNET_MINT ||
    *mint == USDT_MAINNET_MINT ||
    // Devnet for testing
    *mint == USDC_DEVNET_MINT
}
```

This allows devnet tokens (which have no value) to be deposited in production deployments.

Impact

Users can exploit the system to mint tUSD/inUSD with fake collateral.

Recommendation

Remove the devnet mint from production code or add environment checks. For example:

```
#[cfg(feature = "devnet")]
const DEVNET_MINTS: &[Pubkey] = &[USDC_DEVNET_MINT];
pub fn is_whitelisted_collateral(mint: &Pubkey) -> bool {
    *mint == USDC_MAINNET_MINT || *mint == USDT_MAINNET_MINT
    #[cfg(feature = "devnet")]
    {
```

```
        || DEVNET_MINTS.contains(mint)

    }

}
```

Status

Resolved

[H-09] programs/treasury_vault/src/lib.rs - Missing account validation allows arbitrary substitution

Description

Multiple functions accept `Account<'info, Fees>`, `Account<'info, RiskParams>`, and `Account<'info, OracleParams>` without validating that they match the vault's stored addresses. Instances include:

- `ctx.accounts.fees` is not checked against `v.fees`:
  - deposit
  - deposit_tusd
  - deposit_inusd
  - deposit_usdy
  - deposit_fobxx
- `ctx.accounts.risk` is not checked against `v.risk`:
  - deposit_usdy
  - deposit_fobxx
  - process_withdrawal
  - withdraw_tusd
  - withdraw_inusd
  - withdraw_to_usdy
  - withdraw_to_fobxx
  - trigger_withdrawal_breaker
- `ctx.accounts.oracles` is not checked against `v.oracles`:
  - rebalance
  - deposit_usdy
  - update_cpi_oracle

Impact

Users can pass fake fee/risk/oracle accounts with favorable parameters.

Recommendation

Implement validation for all parameter accounts.

Status

Resolved

[H-10] programs/treasury_vault/src/lib.rs#process_withdrawal - Missing user authorization check

Description

The `process_withdrawal` function does not verify that the signer matches the withdrawal request's user:

```
#[derive(Accounts)]

pub struct ProcessWithdrawal<'info> {

    //...

    #[account(mut, close = user)]

    pub withdrawal_request: Account<'info, WithdrawalRequest>,

    #[account(mut)]

    pub user: Signer<'info>,
```

Any user can process another user's withdrawal request on behalf of that user.

Impact

Theft of withdrawal proceeds.

Recommendation

Implement validation to ensure the signer matches `WithdrawalRequest.user`.

Status

Resolved

[H-11] programs/treasury_vault/src/lib.rs#rebalance - Unvalidated Oracle account in ctx.remaining_accounts

Description

The `rebalance` function reads oracle prices from `remaining_accounts` without validation:

```
let ousg_px_fp: u128 = if ctx.remaining_accounts.len() > 0 {

    let pyth_oracle = &ctx.remaining_accounts[0];
```

This is problematic because there is no validation that the account is the intended Pyth oracle account.

Impact

Attackers can pass a fake Oracle account to manipulate the prices used in NAV calculations, potentially enabling value extraction.

Recommendation

Validate that the Oracle account is the Pyth program ID.

Status

Resolved

[H-12] programs/treasury_vault/src/lib.rs#rebalance - Arbitrary token mints in Jupiter swaps

Description

In the `rebalance` function's auto-optimization, the USDC and USDY mints are passed but not validated:

```
match jupiter_v6_cpi::swap_usdc_to_usdy(
    &ctx.accounts.jupiter_program,
    &ctx.accounts.vault_signer,
    &ctx.accounts.vault_usdc_account,
    &ctx.accounts.vault_usdy_account,
    &ctx.accounts.usdc_mint,
    &ctx.accounts.usdy_mint,
```

This allows attackers to pass arbitrary mints to swap vault tokens for worthless tokens.

Impact

Theft of vault assets through fake token swaps, causing the vault to trade valuable assets for worthless ones.

Recommendation

Validate the `usdc_mint` and `usdy_mint` mints before swapping.

Status

Resolved

[H-13] programs/treasury_vault/src/lib.rs#InitializeTreasuryVault - Non-PDA vault account structure

Description

The `Vault` account is initialized as a regular account, not a PDA:

```
#[derive(Accounts)]

pub struct InitializeTreasuryVault<'info> {

    #[account(init, payer = authority, space = 8 + std::mem::size_of::<Vault>())]

    pub vault: Account<'info, Vault>,
```

This allows multiple vaults with the same configuration.

Additionally, the `Fees`, `RiskParams`, and `OracleParams` are also vulnerable to the issue.

Impact

Similar to H-01, this allows anyone to create arbitrary vault/fees/risk params/oracles accounts and pass them to the program to bypass validations via crafted inputs.

Recommendation

Update the vault, fees, risk params, and oracles to be PDAs by specifying the `seeds` and `bump` constraint.

Status

Resolved

[H-14] programs/treasury_vault - Missing signer validation in CPI contexts

Description

Multiple CPI context structs do not include signer validation:

- `lib.rs`
  - OndoUsdcToUsdy
  - OndoUsdyToUsdc
- `drift_oracle.rs`
  - ValidateOracle
- `drift_position_manager.rs`
  - ManageDriftPosition
  - DriftPlacePerpOrderWithOracle
- `jupiter_real_cpi.rs`
  - JupiterSwapCPI
- `oracle_manager.rs`
  - PerformOracleHealthCheck

Impact

Depending on the actual CPI implementation, bypassing access controls may result in the loss of funds.

For example, setting `ctx.accounts.ondo_treasury` to the attacker's address in `OndoUsdcToUsdy` allows the attacker to steal funds from the `ctx.accounts.vault_usdc_account`.

Recommendation

Implement authority checks via Anchor constraints to ensure that privileged entry points can be executed only by legitimate admins.

Status

Resolved

Medium

[M-01] programs/treasury_vault/src/lib.rs#deposit, deposit_tusd, deposit_inusd - Division before multiplication

Description

The `deposit`, `deposit_tusd`, and `deposit_inusd` functions calculate USD value by performing division before multiplication, which can cause precision loss:

```
let usd_value_6_decimals = to_u64_floor(amount_usd_fp / FP * 1_000_000)?;
```

Impact

Potential loss of precision during fee calculations.

Recommendation

Perform multiplication before division to preserve precision.

Status

Resolved

[M-02] programs/treasury_vault/src/lib.rs#execute_usdc_to_usdy_conversion,execute_usdy_to_usdc_conversion - Using saturating subtraction for balance updates

Description

Vault balance update functions use saturating arithmetic instead of checked arithmetic:

```
// In execute_usdc_to_usdy_conversion

vault.collateral_balance = vault.collateral_balance.saturating_sub(amount);

// In execute_usdy_to_usdc_conversion

vault.usdy_balance = vault.usdy_balance.saturating_sub(amount);
```

This is problematic because saturating subtraction silently clamps to zero on underflow instead of failing.

Impact

Silent underflow could mask accounting bugs.

Recommendation

Use checked arithmetic for explicit error handling.

Status

Resolved

[M-03] programs/treasury_vault/src/lib.rs - Hardcoded price and unimplemented integrations

Description

Across the codebase, multiple critical functions contain TODO comments indicating that Oracle integration is unimplemented. Some of these are:

- `collateral_to_usd_fp`: uses hardcoded $100 SOL price instead of Pyth oracle:
  - `// PRODUCTION: Pass Pyth SOL/USD account via remaining_accounts`
  - `// TODO: Pass oracle account in Context`
- `get_usdy_nav_per_token`: returns fallback value without oracle:
  - `// For now, return static value until Oracle integration is restored`
- `get_usdy_nav_per_token_with_oracles`: Oracle integration commented out:
  - `// TODO: Implement oracle integration when oracle module is restored`
- `update_price_feeds`: the function is a stub:
  - `// PRODUCTION: Implement oracle price feed updates using pyth_oracle module`
- `jupiter_swap_usdc_to_usdy`: third-party integrations are not implemented:
  - `Real implementation would call Jupiter API and execute swap`
  - `For now, return simulated output amount`

Impact

- Price calculations use stale or hardcoded values instead of real market data.
- Attackers can potentially exploit price discrepancies between hardcoded values and actual market prices.

Recommendation

Complete the functionalities before production.

Status

Acknowledged

Low

[L-01] programs/treasury_vault/src/lib.rs - Unnecessary mutable markers on accounts

Description

Several account contexts mark accounts as mut when they don't need write access:

- `UpdateFees`: `fees` marked mut.
- `UpdateRisk` and `GuardedAction`: `risk` marked mut.
- `UpdateOracles`: `oracles` marked mut.
- `UpdateCpiOracle`: `vault` marked mut.

Impact

Misleading for code readers and may lead to incorrect assumptions about the codebase.

Recommendation

Remove `mut` from accounts that are only read.

Status

Acknowledged

[L-02] programs/treasury_vault/src/lib.rs#configure_auto_optimization - Unused minimum deposit parameter

Description

The `min_deposit_for_optimization` parameter is set to the `Vault`, but is never used. No function checks this value before executing optimizations.

Impact

Misleading configuration and potentially incorrect logic.

Recommendation

Implement the `min_deposit_for_optimization` parameter accordingly.

Status

Acknowledged

QA

[Q-01] programs/treasury_vault/src/lib.rs#initialize_vault - Code quality improvements

Description

In `initialize_vault`, the current time is fetched twice via `Clock::get()?.unix_timestamp` in lines 527 and 557. It is possible to create a variable and reuse the value.

Recommendation

Declare `Clock::get()?.unix_timestamp` as a constant and use it in lines 527 and 557. Additionally, run `cargo clippy` to increase code clarity and readability.

Status

Acknowledged

Centralisation

The NOTE Protocol project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the NOTE Protocol project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au